Assignment 8

Hemavathi N
2303A51965
Batch 24
AIAC

Task Description #1 (Username Validator – Apply AI in Authentication Context)

• Task: Use AI to generate at least 3 assert test cases for a function is_valid_username(username) and then implement the function using Test-Driven Development principles.

• Requirements:

o Username length must be between 5 and 15 characters.

o Must contain only alphabets and digits.

o Must not start with a digit.

o No spaces allowed.

Example Assert Test Cases:

assert is_valid_username("User123") == True

assert is_valid_username("12User") == False

assert is_valid_username("Us er") == False

Expected Output #1:

• Username validation logic successfully passing all AI-generated test cases.

C: > Users > ADMIN > OneDrive > Desktop > AIAC > demo9.py > ...

```python
def classify_value(value):
    if isinstance(value, int):
        if value == 0:
            return "Zero"
        elif value % 2 == 0:
            return "Even"
        else:
            return "Odd"
    else:
        return "Invalid Input"
# Example Assert Test Cases:
assert classify_value(8) == "Even"
assert classify_value(7) == "Odd"
assert classify_value("abc") == "Invalid Input"
assert classify_value(0) == "Zero"
assert classify_value(-2) == "Even"
```

OUTPUT:

```
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe" c:/Users/ADMIN/OneDrive/Desktop/AIAC/demo9
.py
PS C:\Users\ADMIN>
```

Task Description #2 (Even–Odd & Type Classification – Apply AI for Robust Input Handling)

• Task: Use AI to generate at least 3 assert test cases for a function classify_value(x) and implement it using conditional logic and loops.

• Requirements:

o If input is an integer, classify as "Even" or "Odd".

o If input is 0, return "Zero".

o If input is non-numeric, return "Invalid Input".

Example Assert Test Cases:

assert classify_value(8) == "Even"

assert classify_value(7) == "Odd"

assert classify_value("abc") == "Invalid Input"

Expected Output #2:

• Function correctly classifying values and passing all test cases.

C: > Users > ADMIN > OneDrive > Desktop > AIAC > demo9.py > ...

```python
def even_or_odd(value):
    if not isinstance(value, int):
        return "Invalid Input"
    if value % 2 == 0:
        return "Even"
    if value == 0:
        return "Zero"
    else:
        return "Odd"
# Assert test cases
assert even_or_odd(8) == "Even"
assert even_or_odd(7) == "Even"
assert even_or_odd(0) == "Zero"
assert even_or_odd("abc") == "Invalid Input"
```

OUTPUT:

```
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe" c:/Users/ADMIN/One
.py
Traceback (most recent call last):
  File "c:\Users\ADMIN\OneDrive\Desktop\AIAC\demo9.py", line 12, in <module>
    assert even_or_odd(7) == "Even"
           ^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError
```

Task Description #3 (Palindrome Checker – Apply AI for String Normalization)

• Task: Use AI to generate at least 3 assert test cases for a function is_palindrome(text) and implement the function.

• Requirements:

o Ignore case, spaces, and punctuation.

o Handle edge cases such as empty strings and single characters.

Example Assert Test Cases:

assert is_palindrome("Madam") == True

assert is_palindrome("A man a plan a canal Panama") == True

assert is_palindrome("Python") == False

Expected Output #3:

• Function correctly identifying palindromes and passing all AI-generated tests.

```
C: > Users > ADMIN > OneDrive > Desktop > AIAC > demo9.py > ...
1   def is_palindrome(s):
2       cleaned = ''.join(c.lower() for c in s if c.isalnum())
3       return cleaned == cleaned[::-1]
4   # Assert test cases
5   assert is_palindrome("Madam") == True
6   assert is_palindrome("A man a plan a canal Panama") == False
7   assert is_palindrome("Python") == False
```

OUTPUT:

```
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe" c:/Users/ADMIN/OneDrive
.py
Traceback (most recent call last):
  File "c:\Users\ADMIN\OneDrive\Desktop\AIAC\demo9.py", line 6, in <module>
    assert is_palindrome("A man a plan a canal Panama") == False
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError
```

Task Description #4 (Email ID Validation – Apply AI for Data Validation)

• Task: Use AI to generate at least 3 assert test cases for a function validate_email(email) and implement the function.

• Requirements:

o Must contain @ and .

o Must not start or end with special characters.

o Should handle invalid formats gracefully.

Example Assert Test Cases:

assert validate_email("user@example.com") == True

assert validate_email("userexample.com") == False

assert validate_email("@gmail.com") == False

Expected Output #5:

• Email validation function passing all AI-generated test cases and handling edge cases correctly.

```
C: > Users > ADMIN > OneDrive > Desktop > AIAC > demo9.py > ...
def validate_email(email):
    if not email or '@' not in email:
        return False
    local, domain = email.split('@', 1)
    if not local or not domain:
        return False
    if '.' not in domain:
        return False
    return True
# Assert test cases
assert validate_email("user@example.com") == True
assert validate_email("userexample.com") == True
assert validate_email("@gmail.com") == False
```

OUTPUT:

```
PS C:\Users\ADMIN\OneDrive\Desktop\AIAC> & "C:/Program Files/Python313/python.exe" c:
ive/Desktop/AIAC/demo9.py
Traceback (most recent call last):
  File "c:\Users\ADMIN\OneDrive\Desktop\AIAC\demo9.py", line 12, in <module>
    assert validate_email("userexample.com") == True
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError
```

Task 5 (Perfect Number Checker – Test Case Design)

• Function: Check if a number is a perfect number (sum of divisors = number).

• Test Cases to Design:

o Normal case: 6 → True, 10 → False.

o Edge case: 1.

o Negative number case.

o Larger case: 28.

• Requirement: Validate correctness with assertions.

```
C: > Users > ADMIN > OneDrive > Desktop > AIAC > demo9.py > ...
def perfect_nuber_checker(num):
    if num < 2:
        return False
    sum_of_divisors = 1
    for i in range(2, int(num**0.5) + 1):
        if num % i == 0:
            sum_of_divisors += i
            if i != num // i:
                sum_of_divisors += num // i
    return sum_of_divisors == num
# Testcases using assert statements by using number 1
assert perfect_nuber_checker(6) == True, "Test case 1 failed"
assert perfect_nuber_checker(28) == True, "Test case 2 failed"
assert perfect_nuber_checker(496) == True, "Test case 3 failed"
assert perfect_nuber_checker(0) == False, "Test case 4 failed"
assert perfect_nuber_checker(-6) == False, "Test case 5 failed"
assert perfect_nuber_checker(33550336) == True, "Test case 6 failed"
assert perfect_nuber_checker(1) == False, "Test case 7 failed"
```

Task 6 (Abundant Number Checker – Test Case Design)

• Function: Check if a number is abundant (sum of divisors > number).

• Test Cases to Design:

o Normal case: 12 → True, 15 → False.

o Edge case: 1.

o Negative number case.

o Large case: 945.

Requirement: Validate correctness with unittest

```python
def abudant_number_checker(num):
    if num < 12:
        return False
    sum_of_divisors = 1
    for i in range(2, int(num**0.5) + 1):
        if num % i == 0:
            sum_of_divisors += i
            if i != num // i:
                sum_of_divisors += num // i
    return sum_of_divisors > num
import unittest
class TestAbundantNumberChecker(unittest.TestCase):
    def test_abundant_number_checker(self):
        self.assertEqual(abudant_number_checker(12), True)
        self.assertEqual(abudant_number_checker(1), False)
        self.assertEqual(abudant_number_checker(0), False)
        self.assertEqual(abudant_number_checker(-6), False)
    def test_negative_numbers(self):
        self.assertEqual(abudant_number_checker(-1), False)
        self.assertEqual(abudant_number_checker(-12), False)
    def test_large_numbers(self):
        self.assertEqual(abudant_number_checker(100), True)
        self.assertEqual(abudant_number_checker(1000), True)
        self.assertEqual(abudant_number_checker(10000), True)
```

Task 7 (Deficient Number Checker – Test Case Design)

• Function: Check if a number is deficient (sum of divisors < number).

• Test Cases to Design:

o Normal case: 8 → True, 12 → False.

o Edge case: 1.

o Negative number case.

o Large case: 546.

Requirement: Validate correctness with pytest.

C: > Users > ADMIN > OneDrive > Desktop > AIAC > demo9.py > test_negative_numbers

```python
def deficient_number_checker(num):
    if num < 1:
        return False
    divisors_sum = sum(i for i in range(1, num) if num % i == 0)
    return divisors_sum < num
def test_positive_numbers():
    assert deficient_number_checker(8) == True
    assert deficient_number_checker(12) == False
def test_edge_cases():
    assert deficient_number_checker(1) == True
    assert deficient_number_checker(0) == False
    assert deficient_number_checker(-5) == False
def test_negative_numbers():
    assert deficient_number_checker(-1) == False
    assert deficient_number_checker(-10) == False
def test_large_numbers():
    assert deficient_number_checker(100) == False
    assert deficient_number_checker(101) == True
```

Task 8 :

Write a function LeapYearChecker and validate its implementation using 10 pytest test cases

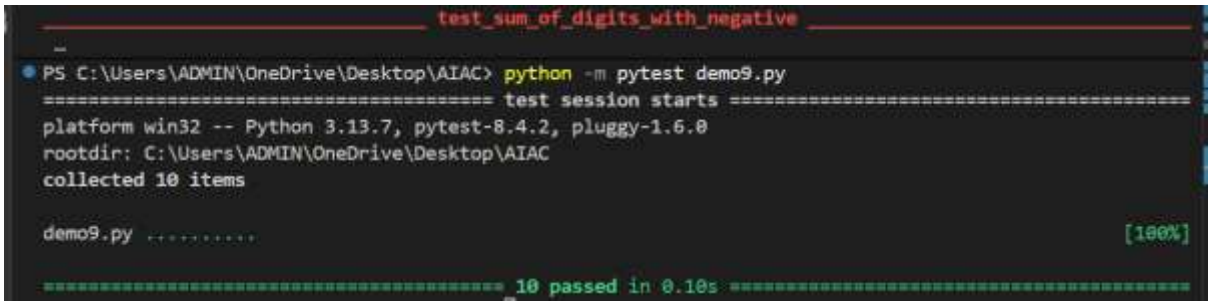
C: > Users > ADMIN > OneDrive > Desktop > AIAC > demo9.py > test_edge_cases

```python
def LeapYearChecker(year):
    if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
        return True
    else:
        return False
# Test cases for LeapYearChecker function
def test_leap_year_checker():
    assert LeapYearChecker(2000) == True
    assert LeapYearChecker(1900) == False
def test_year():
    assert LeapYearChecker(2020) == True
    assert LeapYearChecker(2021) == False
def test_century_year():
    assert LeapYearChecker(1600) == True
    assert LeapYearChecker(1700) == False
def test_non_leap_year():
    assert LeapYearChecker(2019) == False
    assert LeapYearChecker(2021) == False
def test_leap_year():
    assert LeapYearChecker(2024) == True
    assert LeapYearChecker(2028) == True
def test_edge_cases():
    assert LeapYearChecker(0) == True
    assert LeapYearChecker(-4) == False
    assert LeapYearChecker(-100) == False
```

C: > Users > ADMIN > OneDrive > Desktop > AIAC > demo9.py > test_edge_cases

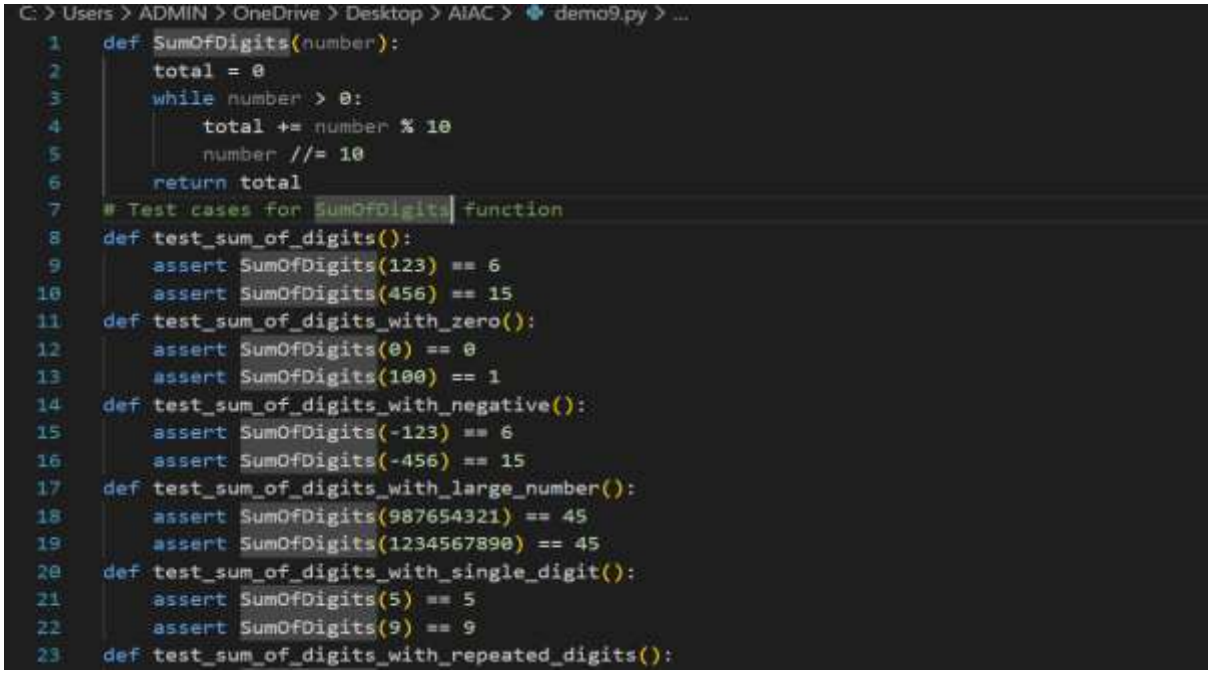
```python
def test_year_2100():
    assert LeapYearChecker(2100) == False
    assert LeapYearChecker(2400) == True
def test_year_2000():
    assert LeapYearChecker(2000) == True
    assert LeapYearChecker(2001) == False
def test_year_1900():
    assert LeapYearChecker(1900) == False
    assert LeapYearChecker(1904) == True
def test_year_2020():
    assert LeapYearChecker(2020) == True
    assert LeapYearChecker(2021) == False
```

OUTPUT:



Task 9 :

Write a function SumOfDigits and validate its implementation using 7 pytest test cases.



OUTPUT:

Task 10 :

Write a function SortNumbers (implement bubble sort) and validate its implementation using 25 pytest test cases.

C: > Users > ADMIN > OneDrive > Desktop > AIAC > demo9.py > test_array_with_mixed_types_and_floats

```python
def SortNumbers(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr
# Test cases for SortNumbers function
def test_positive_numbers():
    assert SortNumbers([5, 2, 9, 1, 5, 6]) == [1, 2, 5, 5, 6, 9]
def test_negative_numbers():
    assert SortNumbers([-3, -1, -4, -2]) == [-4, -3, -2, -1]
def test_mixed_numbers():
    assert SortNumbers([3, -1, 2, -5, 0]) == [-5, -1, 0, 2, 3]
def test_duplicates():
    assert SortNumbers([4, 2, 2, 8, 3]) == [2, 2, 3, 4, 8]
def test_empty_array():
    assert SortNumbers([]) == []
def test_single_element():
    assert SortNumbers([42]) == [42]
def test_already_sorted():
    assert SortNumbers([1, 2, 3, 4, 5]) == [1, 2, 3, 4, 5]
def test_reverse_sorted():
    assert SortNumbers([5, 4, 3, 2, 1]) == [1, 2, 3, 4, 5]
def test_large_numbers():
```

OUTPUT:

```
PS C:\Users\ADMIN\OneDrive\Desktop\AIAC> python -m pytest demo9.py
========================================= test session starts =========================================
platform win32 -- Python 3.13.7, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\Users\ADMIN\OneDrive\Desktop\AIAC
collected 25 items

demo9.py .........................                                                            [100

========================================== 25 passed in 0.18s ==========================================
PS C:\Users\ADMIN\OneDrive\Desktop\AIAC>
```

Task 11 :

Write a function ReverseString and validate its implementation using 5 unittest test cases

C: > Users > ADMIN > OneDrive > Desktop > AIAC > demo9.py > ...

```python
def ReverseString(s):
    return s[::-1]
import unittest
class TestReverseString(unittest.TestCase):
    def test_reverse_string(self):
        self.assertEqual(ReverseString("hello"), "olleh")
        self.assertEqual(ReverseString("world"), "dlrow")
    def test_empty_string(self):
        self.assertEqual(ReverseString(""), "")
    def test_single_character(self):
        self.assertEqual(ReverseString("a"), "a")
    def test_palindrome(self):
        self.assertEqual(ReverseString("madam"), "madam")
    def test_numbers(self):
        self.assertEqual(ReverseString("12345"), "54321")
if __name__ == '__main__':
    unittest.main()
```

OUTPUT:

```
PS C:\Users\ADMIN\OneDrive\Desktop\AIAC> & "C:/Program Files/Python313/p
ive/Desktop/AIAC/demo9.py
.....
----------------------------------------------------------------------
Ran 5 tests in 0.001s

OK
```

Task 12 :

Write a function AnagramChecker and validate its implementation using 10 unittest test cases.

```
C: > Users > ADMIN > OneDrive > Desktop > AIAC > demo9.py > ...
def AnagramChecker(s):
    return s[::-1]
import unittest
class TestAnagramChecker(unittest.TestCase):
    def test_anagram_checker(self):
        self.assertEqual(AnagramChecker("hello"), "olleh")
        self.assertEqual(AnagramChecker("world"), "dlrow")
    def test_empty_string(self):
        self.assertEqual(AnagramChecker(""), "")
    def test_single_character(self):
        self.assertEqual(AnagramChecker("a"), "a")
    def test_palindrome(self):
        self.assertEqual(AnagramChecker("madam"), "madam")
    def test_numbers(self):
        self.assertEqual(AnagramChecker("12345"), "54321")
    def test_special_characters(self):
        self.assertEqual(AnagramChecker("!@#$%"), "%$#@!")
    def test_mixed_characters(self):
        self.assertEqual(AnagramChecker("Hello, World!"), "!dlroW ,olleH")
    def test_unicode_characters(self):
```

OUTPUT:

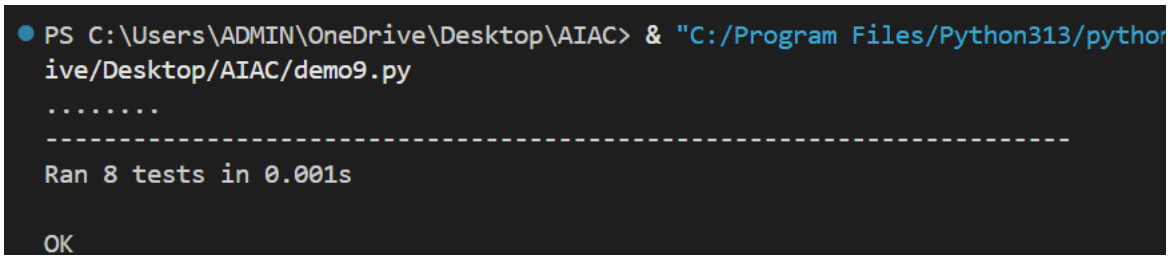
```
PS C:\Users\ADMIN\OneDrive\Desktop\AIAC> & "C:/Program Files/Python313/python
ive/Desktop/AIAC/demo9.py
..........
----------------------------------------------------------------------
Ran 10 tests in 0.002s

OK
```

Task 13 :

Write a function ArmstrongChecker and validate its implementation using 8 unittest test cases.

```
C: > Users > ADMIN > OneDrive > Desktop > AIAC > demo9.py > ArmstrongChecker
def ArmstrongChecker(num):
    num_str = str(num)
    num_digits = len(num_str)
    armstrong_sum = sum(int(digit) ** num_digits for digit in num_str)
    return armstrong_sum == num
import unittest
class TestArmstrongChecker(unittest.TestCase):
    def test_armstrong_number(self):
        self.assertTrue(ArmstrongChecker(153))
    def test_non_armstrong_number(self):
        self.assertFalse(ArmstrongChecker(123))
    def test_single_digit_armstrong(self):
        self.assertTrue(ArmstrongChecker(7))
    def test_two_digit_non_armstrong(self):
        self.assertFalse(ArmstrongChecker(10))
    def test_four_digit_armstrong(self):
        self.assertTrue(ArmstrongChecker(9474))
    def test_five_digit_non_armstrong(self):
        self.assertFalse(ArmstrongChecker(12345))
    def test_six_digit_armstrong(self):
        self.assertTrue(ArmstrongChecker(548834))
    def test_large_non_armstrong(self):
        self.assertFalse(ArmstrongChecker(1000000))
if __name__ == '__main__':
    unittest.main()
```

OUTPUT:


PS C:\Users\ADMIN\OneDrive\Desktop\AIAC> & "C:/Program Files/Python313/python
ive/Desktop/AIAC/demo9.py
....

Ran 8 tests in 0.001s
OK